Serving WebDAV

Implementing a WebDAV Server using PHP 4.3 and PEAR.

Hartmut Holzgraefe

Web Distributed Authoring and Versioning - WebDAV - is a HTTP protocol extension designed to replace FTP for website maintenance. After a rather slow start it is now widely available in server and client products. So it is about time to make our favourite language support it.

Even back in 1996 it was obvious that the usual file transfer tools where not really suitable for publishing processes in the emerging World Wide Web, so the Web Publishing Working Group was founded. In February 1998 it published its findings as a conceptual paper in RFC 2291. One year later the current WebDAV protocol standard was defined and specified in RFC 2518. This document describes only the WebDAV part of the protocol, the specification process of the versioning part of the protocol took some more time and was finally published three years later as RFC 3253. Most WebDAV servers and clients these days still support RFC 2518 only.

The first reference implementation of RFC 2518 was the apache module mod_dav which is still widely used. Microsoft IIS and Apple MacOS/X also offer the possibility to remotely maintain files using WebDAV. But WebDAV is not limited to remote file access alone, it is also offered by some content management systems as a way to access and maintain their templates and content data using general clients, and it is even the protocol used between Microsoft Exchange and its Outlook Clients.

On the Client side you can access WebDAV servers just as remote file systems on the common operation system platforms Windows (starting with Windows98 and Windows2000, with the WindowsXP client having some serious bugs), MacOS/X and Linux. More specific control is provided by special WebDAV clients and by authoring tools like e.g. Macromedia DreamWeaver.

When the Web Publishing Working Group was founded the usual way to maintain a web site was via FTP, which was considered to be too insecure and too complicated for general use. Network file systems like NFS or SMB that allowed for direct file system access where platform specific and not optimised for slow network connections HTTP itself provided the PUT method for uploadding files but no standard way of getting directory listings containing file names, sizes and access permissions (you may get directory listings on some servers for directories where no default document like index.html is present, but this is not specified by the HTTP standard and there is no fixed format for these). Products like FrontPage tried to solve the problem with special CGI scripts using their own protocol but were considered being too proprietary.

With WebDAV HTTP is extended by new methods in addition to the already existing GET and PUT. MKCOL, COPY, MOVE and DELETE provide file system manipulation, PROPFIND and PROPSET give read and write access to meta information and LOCK and UNLOCK offer shared and exclusive protection of work in progress. The DeltaV extensions as specified in RFC 3252 add further methods for versioning, change management and archiving, which are beyond the focus of this article.

The WebDAV Protocol

A WebDAV server manages collections, resources and properties. Collections are just like directories or folders in usual file systems and resources correspond to files stored in a file systems. Properties are assigned to either collections or resources and may contain any kind of additional information (similar to the resource forks in Macintosh file systems). The collection hierarchy of a WebDAV Server and the resources contained within are unambiguously mapped to HTTP URLs. Some standard properties are defined by RFC 2518, but arbitrary properties

may be added at any time using namespaces. The difference in naming conventions between WebDAV and file systems is intended, as WebDAV is not only suitable for accessing remote files, but any kind of data organized in a tree-like form.

WebDAV resources are stored and retrieved using the HTTP *PUT* and *GET* methods (the result of a *GET* request for a WebDAV collection is not defined, but it is usual to present some sort of directory listing in this case). Collection contents are manipulated using *MKCOL* (which creates a new sub-collection), *COPY*, *MOVE* and *DELETE*. WebDAV properties are retrieved and set using the *PROPFIND* and *PROPSET* methods.

When requesting a *PROPFIND* on a collection it is possible to add a "Depth:" header that specifies if you are interested in the properties of the collection alone (Depth: 0), those of the collection and the resources and collections it contains itself (Depth: 1) or a complete recursive listing of all subsequent resources and collections contained (Depth: Infinity). A *PROPFIND* with a depth greater zero is the only way to identify a collections contents.

Simple WebDAV methods like *GET*, *PUT* and *DELETE* get all the needed control information from the HTTP request header, whereas the more advanced methods like *PROPFIND* and *PROPSET* are controlled by a mix of HTTP headers (like "Depth:") and additional XML formatted information in the request body. WebDAV is completely XML namespace aware as namespaces are needed to avoid conflicts in property names and content, as properties can store arbitrary XML data. WebDAV itself defines the special reserved "*DAV:*" namespace for its own tags and properties.

Unfortunately general property access is not supported by the remote file system clients provided by common operating systems. Microsoft Web Drives, the MacOS/X finder and Linux davfs do not care about any properties besides those defined by the WebDAV specification for file system meta data like size or access date. Especially the MacOS/X implementation is disappointing here as it is not capable of mapping Macintosh file system resources into WebDAV properties but insists on storing them in hidden files.

Complete property access is provided by the unix command line tool 'cadaver' and the java client SkunkDAV, limited access is provided by the Windows client TeamDrive. When using WindowsXP you should also consider a tool like TeamDrive as the XP WebDAV client implementation still has some serious flaws (which were not present in the Windows2000 client).

The WebDAV specification in RFC 2518 is unfortunately not as clear and explicit as other IETF standard documents. This leads to open questions when trying to implement a compliant server. In case of doubt it is always possible to consult the reference implementation mod_dav, and the (inofficial) *litmus* WebDAV server protocol compliance test may also help. But the overall situation is still not very satisfying and the 'quality' of some common clients does all but improve this. Especially Microsoft, although itself heavily involved in the creation of the specification, is known for its rather arbitrary interpretation of the standard. So to create a server playing nice with common clients you not only have to follow the specifications but also need to analyse clients behaviour using tools like HTTP protocol debuggers.

Some examples:

- Microsoft IIS allows the creation of web folders on the document root level only. Consequently Microsoft clients believe that this is true for any WebDAV server
- Microsoft clients may change URL parts to all uppercase at will. Only after a 404 error is received they will fall back to the URL as entered by a user
- MacOS/X and Macromedia DreamWeaver presume lock support on any server, even if the server does not claim to support locking
- MacOS/X may try to retrieve a hidden resource file up to eight or even sixteen times in a row until it believes a "404 not found" reply by a server

Help provided by PEAR

The PEAR class HTTP_WebDAV_Server provides a framework to spare you some of these troubles in implementing a WebDAV server. HTTP conversation itself, authentication and encryption are already managed on the web server and the PHP level. The server class builds upon this and implements the general parts of the WebDAV protocol like XML parsing and generation. This lets you focus on the mapping of your applications objects to collections and resources right away.

HTTP_WebDAV_Server is available in the PEAR repository. It provides the generic server base class and a sample file server reference implementation. The base class is still experimental and work-in-progress, but its general architecture and interfaces are already stable. The fileserver sample passes all tests within the litmus compliance test suite, with the only exception of locking tests. Locking support is provided by the base class but not yet used in the file server sample.

The base class defines the abstract methods *GET(), PUT(), DELETE(), PROPFIND(), PROPPATCH(), MKCOL(), COPY(), MOVE(), LOCK(), UNLOCK(), CHECKLOCK()* and *AUTH()*. Most of these match the WebDAV methods by the same name, only *CHECKLOCK()* and *AUTH()* are needed for more general tasks.

A minimal server would only have to implement *GET(), PUT()* and *PROPFIND()*, although this server would not be RFC 2518 compliant. A compliant server would also need *MKCOL(), DELETE()* and *PROPPATCH(). MOVE()* and *COPY()* are executed using *GET(), PUT()* and *DELETE()* if not implemented, although this might not give the same performance as if you would have written your own implementations. *LOCK(), UNLOCK()* and *CHECKLOCK()* are only needed for WebDAV Level 2 conformance. The *AUTH()* method is not related to WebDAV itself but to the authentication scheme you want to implement (you should never provide WebDAV access to a server without authentication).

All these methods expect their parameters in a single parameter array that is passed by reference. Only *PROPFIND()* will be called with a second parameter containing an array describing the requested properties.

The actual access to your application data needs to be implemented using *GET()* and *PUT()*. A WebDAV GET request is like a normal HTTP request that returns the content data for some resource identified by its URL. The *GET()* method needs only a single parameter, the requested resource path in *$params['path']*. It has to return its HTTP status in *$param['status']*, for a successful request it also has to specify a MIME type in *$params['type']*, the number of bytes written in *$params['size']* and a modification date as a unix timestamp in *$params['last_mod']*. The actual data is written as usual with *echo(), readfile()* or any other standard output function. Possible status values are 200 for success, 404 for nonexisting resources and 409 for other problems.

```
// sample implemetation for filesystem GET
function GET(&$params)
{
  $file = $this->document_root . $params["path"];
  if(!file_exists($file)) {
    $params['status'] = 404; // not found
    return;
  }
  if(!is_readable($file)) {
    $params['status'] = 409; // conflict
    return;
  }
  $params['status'] = 200; // Ok
  $params['size'] = filesize($file);
  $params['last_mod'] = filemtime($file);
  $params['type'] = "application/octet-stream";
  readfile($file);
}
```

The *PUT()* methods gets the resource path in *$params['path']*, the number of transferred data bytes in *$params['size']*, the MIME type in *$params['type']* and a PHP stream resource to read from in *$params['stream']*. It has to return a HTTP status in *$params['status']*, possible values are 201 for a newly created resource, 204 for successful updates, 507 if the resource exceeds size limitations and 409 for any other problem.

```
// sample implementation for filesystem PUT
```

```
function PUT(&$params)
{
  $file = $this->document_root . $params["path"];
  $exists = file_exists($file);
  $out = fopen($file, "w");
  if (is_resource($out)) {
    $in = $params['stream'];
    while(!feof($in)) {
      fwrite($out, fread($in, $params['size']));
    }
    fclose($out);
    $params['status'] =
      $exists ? 204 /* exists */
              : 201 /* created */;
  } else {
    $params['status'] = 409; // conflict
  }
}
```

The *DELETE()* method gets the resource path in *$params['path']* and the requested recursion depth in *$params['depth']*. Possible depth values are 0 and '*Infinity*'. A WebDAV delete is an atomic operation, it must succeed or fail completely, a partial deletion is not supported by the specification. A successful deletion has to return a HTTP 204 status in *$prefs['status']*, a failed delete has to return an array of pathname / status pairs in *$params['files']* instead.

```
// sample implementation for filesystem DELETE
function DELETE(&$params) {
  $file = $this->document_root . $params["path"];
  system("rm -f $file");
  $params["status"] = 204;
}
```

New collections (directories) are created using the *MKCOL()* method. It gets the resource path in *$param['path']* and has to return its HTTP status in *$params['status']*. Possible values are 204 for success and 409 in case of any problems.

```
// sample implementation for filesystem MKCOL
function MKCOL(&$params)
{
  $file = $this->document_root . $params["path"];
  if(file_exists($file) || !is_dir(dirname($file)) {
    $params['status'] = 409;
  } else if(mkdir($file)) {
    $params['status'] = 204;
  } else {
    $params['status'] = 409;
  }
}
```

Now you have all the pieces for a first WebDAV server in place. *COPY()* and *MOVE()* will for now be emulated as mentioned above. This implementation is not yet standards compliant, but it is already complete enough to deal with WebDAV file system clients.

Who's that knocking at my door?

You should never run a WebDAV service without authentication, as data on the server may be modified using WebDAV requests. You can protect your server script using standard HTTP access restrictions as provided by your webserver (e.g. .htaccess files).

For webserver APIs that support PHP authentication you can also deal with authentication issues from within your server application. In this case you have to implement the *AUTH()* method. *AUTH()* will be called with username, password and authentication method as parameters. You have to return true for a successful authentication, otherwise you have to return the realm string to be displayed in the clients login dialog.

```
// sample implementation for authentication check
function AUTH($user, $passwd, $method)
{
  if($user == "foo" && $passwd == "bar")
    return true;
  return "FooBar Login";
}
```

Most WebDAV clients do not accept cookies and you have no *GET* parameters with WebDAV, so you can't use PHP sessions unless you come up with your own scheme to create unique session IDs. You can use the username as session id if you do not plan to support concurrent sessions for a single user. For concurrent session support you may try to combine user

name and the clients IP address or User-Agent string, but you have to be aware of the fact that the IP may change within a session due to DHCP or proxy issues and that for some clients the user agent identification may change during a session depending on the request context

Properties

Properties give you the possibility to store arbitrary meta information in WebDAV properties is similar or even superior to mechanisms like MacOS resource forks. It is a very powerful, but also a very complex mechanism that makes heavy use of XML including namespaces (and has some client specific gotchas, too). The server base class tries to encapsulate as much of the needed functionality as possible by providing XML generation and parsing. All you have to deal with are PHP arrays containing property information, but still you will find that your *PROPFIND()* and *PROPPATCH()* are more complex then the other methods.

The *PROPFIND()* method gets the path of the queried resource or collection in *$params ['path']*, the recursion depth for collections in *$params['depth']* (0, 1 or '*infinity*') and the requested properties in *$params['props']*. There are two special cases in which the clients wants to get all property values or just the names of all existing properties. In these cases *$params ['props']* is just a string "all" or "names". The more common case is that a client asks for some specific properties. Here *$params['props']* will contain an array of property descriptions which themselves are arrays containing two elements '*name*' and '*xmlns*' for the property name and XML name space.

PROPFIND() has to return its execution status in *$params['status']* and the found properties in the second referential parameter *$files*. Possible status values are 207 for success (even partial) or 404 if the requested path was not found.

The result array *$files* has to be filled with one entry per found resource or collection. Each file entry is itself an array with the elements '*path*' containing the complete resource or collection path and 'props' containing an array of property descriptions. A property description entry is either a simple key/value pair with the name of a WebDAV property as the key and its value as value or an array with the elements '*name*', '*xmlns*' and '*val*' for a complete property description. The two following entries are equivalent:

```
// short form for DAV: namespace only
$file["props"]["displayname"] = "foobar";

// long form including namespace
$file["props"][] =
  array ("name"  => "displayname",
         "xmlns" => "DAV:",
         "val"   => "foobar"
        );
```

RFC 2518 defines a set of standard WebDAV properties in the *DAV:* name space. You should provide at least the following:

property name	format	description
resourcetype	string	"collection" or empty
getcontentlenght	int	content size in bytes
getcontenttype	string	content MIME type
creationdate	unix timestamp	resource creation date
getlastmodified	unix timestamp	resource modification date
displayname	string	optional: name to display instead of path name

You may always return all properties without looking at the requested set in *$params ['props']* as the base class will take care to use only those entries in *$files* that were requested by the client. But be warned that you may waste a lot of memory and CPU time this way.

```
// sample implementation for getting filesystem properties
function propfind (&$params, &$files)
{
  $file = $this->document_root . $params["path"];
  if (!file_exists($file)) {
    $params['status'] = 404;
```

```
    return false;
  }
  $files = array();
  $files[] = $this->fileinfo($file, $params);
  if (!empty($params["depth"])) {
    if (substr($params["path"], -1) != "/") {
      $params["path"] .= "/";
    }
    $handle = opendir($fspath);
    while ($filename = readdir($handle)) {
      if ($filename != "." && $filename != "..") {
        $fullpath = ($params["path"] . $filename;
        $files["files"][] =
          $this->fileinfo($fullpath, $params);
      }
    }
  }
  return true;
}

function fileinfo($uri, $params)
{
  $fspath = $this->base . $uri;
  $props = array();
  $props["displayname"] =
    strtoupper($uri);
  $props["creationdate"] =
    filectime($fspath);
  $props["getlastmodified"] =
    filemtime($fspath);
  if (is_dir($fspath)) {
    $props["getcontentlength"] = 0;
    $props["resourcetype"] = "collection";
    $props["getcontenttype"] =
      "httpd/unix-directory";
  } else {
    $props["resourcetype"] = "";
    $props["getcontentlength"] =
      filesize($fspath);
    if (is_readable($fspath)) {
      $props["getcontenttype"] =
        rtrim(preg_replace("/^([^;]*);.*/", "$1",));
    } else {
      $props["getcontenttype"] =
        "application/x-non-readable";
    }
  }

  return array("path" => $uri,
               "props"=> $props
               );
}
```

PROPPATCH() gets the resource path in *$params['path']* and an array of properties to set or modify in *$params['props']*. The property elements are of the same format as those returned by *PROPFIND()*, consisting of the three elements '*name*', '*xmlns*' and '*val*' each. An entry without '*val*' (unset, not just empty) is a request to delete this property. Note that some properties exist without being set by *PROPPATCH()*, and that some of these aren't even modifiable by *PROPPATCH()*. These properties, like '*getcontentlength*', are called 'life' properties while those that are modified by *PROPPATCH()* only are called 'dead' properties.

PROPPATCH() returns its execution status in *$params['status']*, possible values are 207 for success (even partial) or 404 if the requested path doesn't exist. Each requested property has a sub-status of its own. In case a property value could not be set or a delete request could not be fulfilled you have to add a '*status*' element to the property array. Possible sub-status values are 403 if the property can't be changed or deleted, 409 if the property value is not suitable to be set or the property is read only or 507 if there is not enough storage space to store the value.

The following example implementation allows to change the 'getlastmodified' property but rejects any other property change request:

```
// sample implementation for setting filesystem properties
function proppatch (&$params)
{
  $file = $this->document_root . $params["path"];
  if (!file_exists($file)) {
    $params['status'] = 404;
  } else {
    foreach($params['props'] as $key => $prop) {
```

```
      if(   $prop['name'] == "getlastmodified"
         && $prop['xmlns'] == "DAV:") {
        touch($file, $prop["value"]);
      } else {
        $params['props'][$key]['status'] = 403;
      }
    }
    $params['status'] = 207;
  }
}
```

Testing - one, two ...

Now you can put all the peaces together to test your server. All you have to do is to extend the base class by adding worker methods for the supported WebDAV request methods as described above and to instantiate a single object of that class. The constructor method will process the request and will delegate work to your methods as needed. No further action is needed to serve the actual request.

```
<?php
  // putting it all together

  // get the PEAR base class
  require_once "HTTP/WebDAV/Server.php";

  // extend it as described above
  class MyWebDAVServer
    extends HTTP_WebDAV_Server
  {
    // your method impelentations go here
    ...
  }

  // now serve the request by creating
  // a server object. the constructor
  // method will do all the work, no
  // further action required
  new MyWebDAVServer;
?>
```

If possible you should test your server implementation with the Unix command line client 'cadaver' first. The user interface of this client is command driven and very similar to the default ftp client on Unix systems, but is known for its very clean and complete implementation of the WebDAV specification. Testing against other clients should only be the second step as this way you get a clue whether an emerging problem is in the clients or the servers implementation.

When aiming at a complete standards compliant implementation you can also use the litmus WebDAV compliance test suite. This is not an 'official' test suite so passing its tests doesn't give you any certification, but nonetheless trying to comply to its requirements is usually worth the effort.

Conclusion

The PEAR HTTP_WebDAV_Server class provides a framework, that eases the implementation of WebDAV server applications using PHP a lot. It can be installed on your system (providing you have a working PEAR setup) by 'pear install HTTP_WebDAV_Server. The class is still 'work in progress', but already capable of providing an WebDAV Level 1 compatible server setup. Preliminary locking support, which is required for WebDAV Level 2 conformance, is also included.

WebDAV capabilites are especially interesting in, but not limited to, the web publishing and content management area. Every application managing hierarchically data may benefit from providing a file system like view using the WebDAV protocol.

Resources

PEAR:

- PEAR *http://pear.php.net/*
- HTTP_WebDAV_Server *http://cvs.php.net/cvs.php/pear/HTTP_WebDAV_Server*

WebDAV Resources:

- WebDAV *http://www.webdav.org/*
- WebDAV Standard RFC 2518 *http://www.ietf.org/rfc/rfc2518.txt*
- WebDAV Versioning RFC 3253 *http://www.ietf.org/rfc/rfc3253.txt*

Free clients:

- `cadaver` commandline client *http://www.webdav.org/cadaver/*
- SkunkDAV Java GUI client http://skunkdav.sourceforge.net/
- Goliath MacOS client *http://www.webdav.org/goliath/*
- `litmus` compliance test suite *http://www.webdav.org/neon/litmus/*

Free server:

- Apache mod_dav *http://www.webdav.org/mod_dav/*

Windows products:

- WebDrive *http://www.webdrive.com/*
- TeamDrive *http://www.teamstream.com/product.htm*

This presenation has ended.
Click anywhere to return.

_

